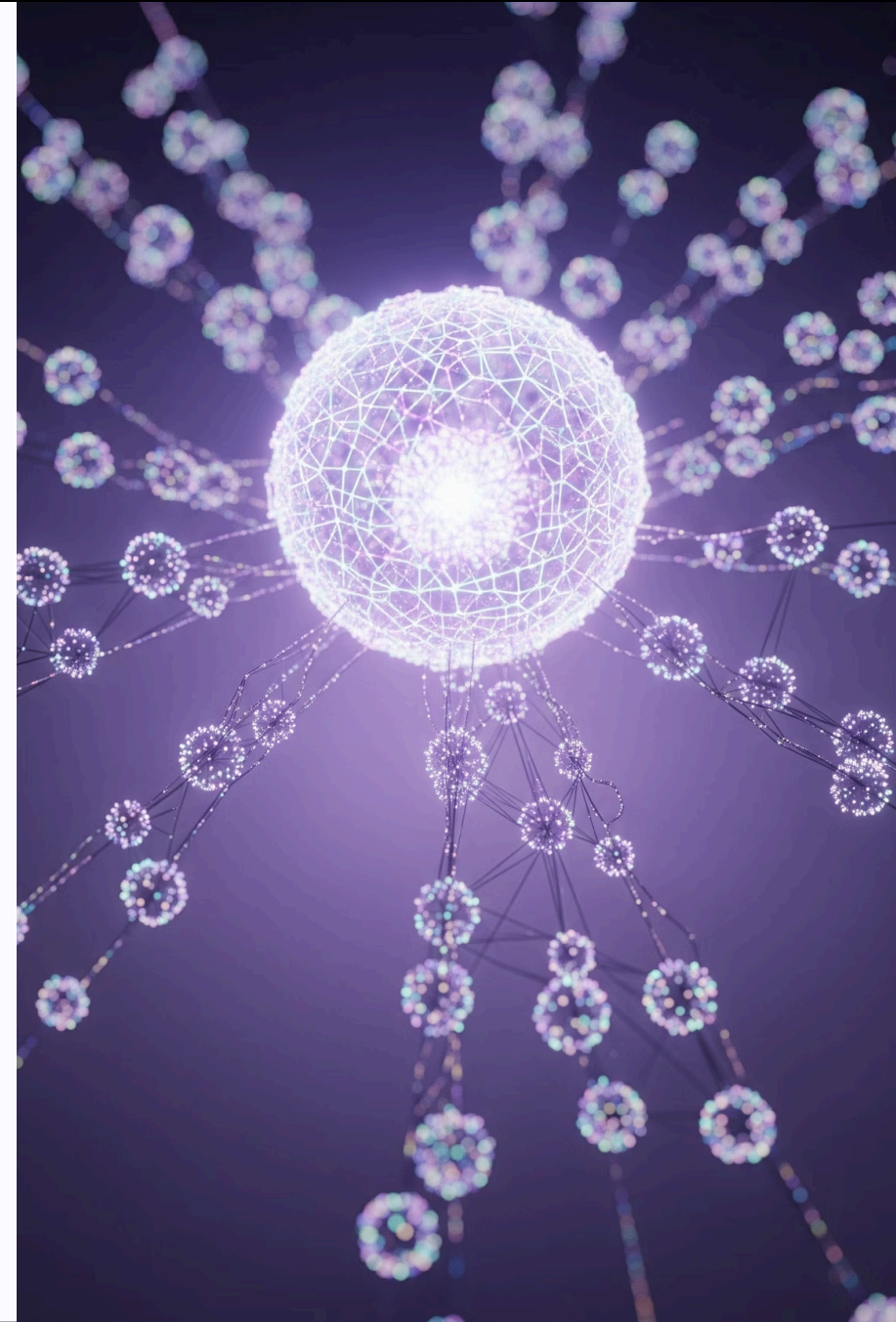


Fine-Tuning vs Parameter-Efficient Tuning (PEFT)

Adapting Large Models Without Breaking the Bank

A practical guide for ML engineers and product managers looking to efficiently adapt transformer models



Why Tuning Matters

Pretrained LLMs provide powerful general capabilities, but real-world applications demand **specialized knowledge** and behaviors:

1

General-Purpose Foundation

Base models trained on broad data lack domain expertise

2

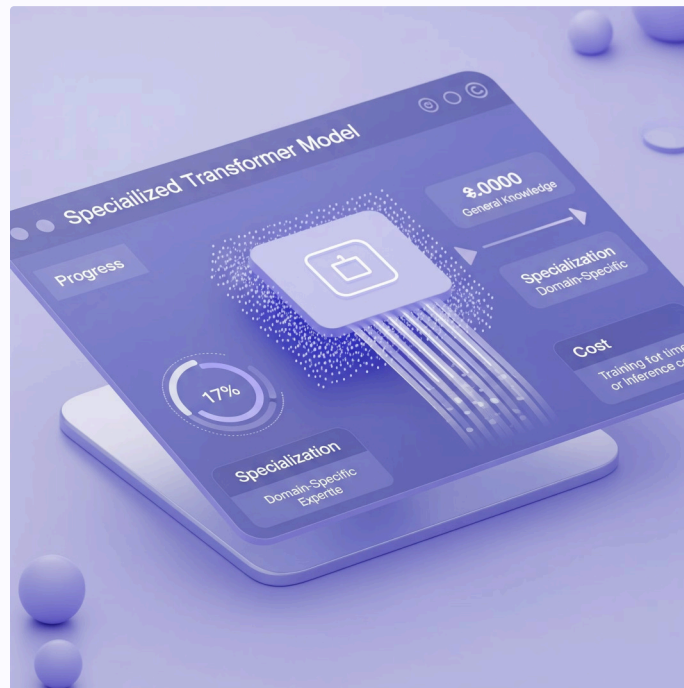
Domain Specialization

Fine-tuning adapts models to specific tasks and knowledge domains

3

Resource Constraints

Full fine-tuning requires massive compute, storage, and training data



PEFT enables adaptation with minimal resources — the focus of this presentation



Full Fine-Tuning

The traditional approach: update **all parameters** of the model

Advantages

- Maximum flexibility in model adaptation
- Highest potential accuracy/performance
- Well-established methodology with proven results
- Full control over model behavior

Limitations

- Requires enormous compute resources (GPUs/TPUs)
- High storage costs (100s of GBs per model)
- Needs substantial training data
- Deployment challenges for multiple specialized versions

Parameter-Efficient Tuning (PEFT)

Instead of updating billions of parameters, PEFT approaches modify only a **tiny fraction** of the model.

Selective Updates

Only tune a small subset of parameters (0.1-3% of total)

Lightweight "Deltas"

Store small weight changes on top of frozen base model

Multi-Task Efficiency

Support multiple domains with one base model + small adaptations

Key PEFT Methods

LoRA (Low-Rank Adaptation)

Adapters (bottleneck layers)

Prefix/Prompt Tuning

P-tuning & BitFit

PEFT Techniques Explained

LoRA

Injects trainable low-rank matrices into attention layers, decomposing weight updates into smaller matrices (rank r) that approximate full updates

Adapters

Adds small bottleneck layers between transformer blocks, creating a bottleneck architecture that reduces parameter count while maintaining performance

Prefix/Prompt Tuning

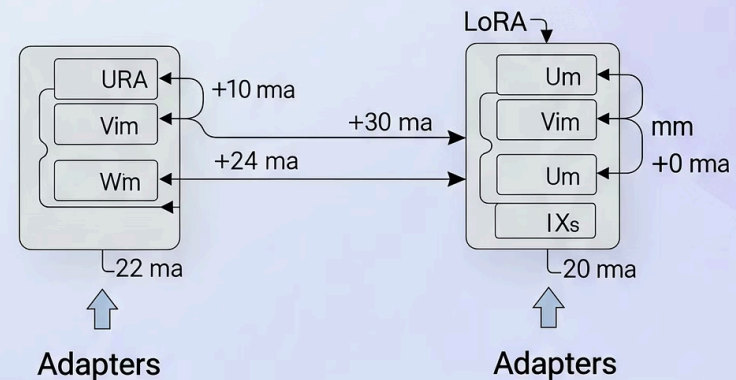
Learns continuous prompt embeddings that are prepended to inputs, effectively "steering" the frozen model toward desired outputs

BitFit

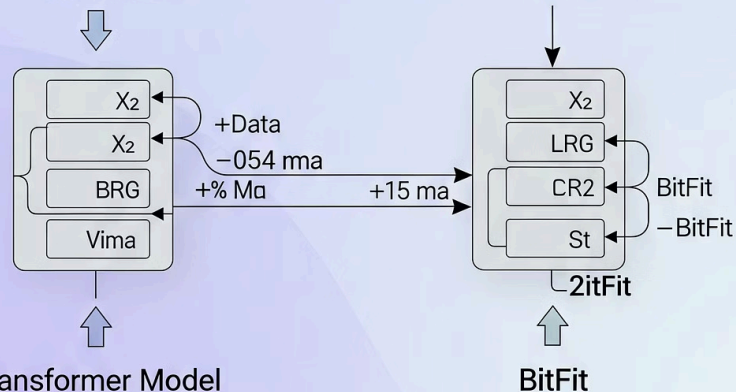
Tunes only bias terms in model weights, leaving matrix weights frozen, targeting the minimal set of parameters for adaptation

Transformer Model

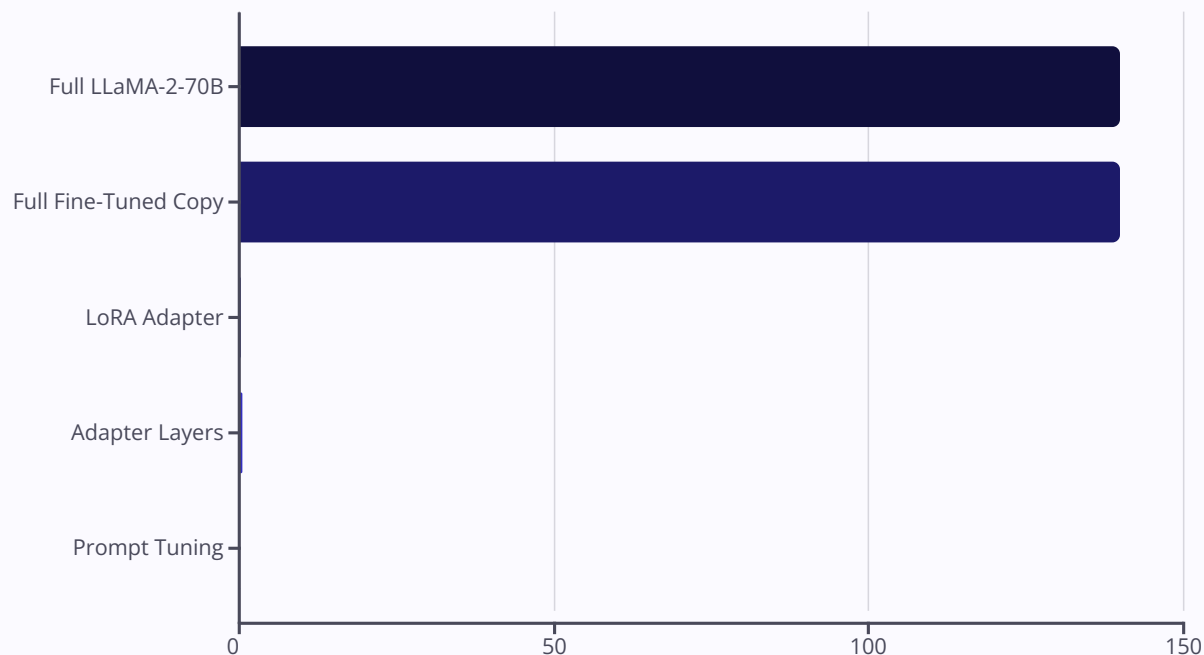
LoRA (Low-Rank Adaptation)



Prefix Tuning



Cost & Storage Comparison



PEFT Advantages

- **10-100× cheaper** training costs
- **10-1000× smaller** storage requirements
- Faster training cycles (hours vs. days/weeks)
- Enables modular, composable model adaptations
- Simpler deployment and version management

A single organization can maintain hundreds of specialized PEFT adapters for the cost of one full fine-tuned model copy.



Use Cases



Domain Adaptation

Specialized models for legal, medical, finance, and scientific domains where vocabulary and reasoning patterns differ from general language



Multilingual Fine-tuning

Adapting models for specific languages or dialects without retraining the entire model for each language variant



On-device Personalization

Lightweight adaptations for personal assistants, chatbots, and copilots that can run on edge devices with limited resources



Rapid Prototyping

Startups and research labs can quickly test specialized models without massive infrastructure investments

Best Practices



Start with PEFT

Begin with LoRA or adapters as your default approach unless you have evidence the task requires deeper changes



Track Trade-offs

Monitor the performance gap between PEFT and full fine-tuning — accept small degradations for massive resource savings



Hybrid Approaches

Combine PEFT with quantization for edge deployment, or with selective layer freezing for optimal performance

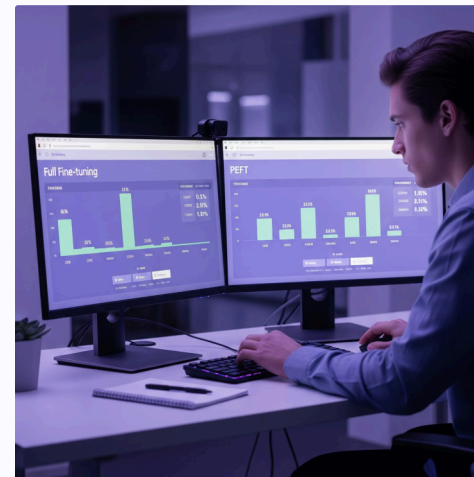
Decision Framework

Use PEFT when: Limited compute/storage, multiple domains needed, rapid iteration required

Consider full fine-tuning when: Critical applications with high performance demands, substantial training data available, and compute resources aren't constrained

The **ideal approach often combines techniques** — base models with interchangeable adapters for different use cases.

Key Takeaways



Next steps: Explore [Hugging Face PEFT](#) library and [Microsoft LoRA](#) implementations to start experimenting with these techniques.